

10-mavzu. Qism dasturlar, funksiya va protseduralar.

Reja

1. Python dasturlash tilida qism dasturlarda funksiyalarni qo'llanilishi.
2. Qism dasturlar va funksiyalar muhandislik va texnik masalalarni yechishda qo'llanilishi.

Kirish

Dasturlash amaliyotida dastur hajmi va murakkabligi oshgani sari uni boshqarish, tushunish va rivojlantirish tobora qiyinlashib boradi. Ayniqsa, muhandislik va texnik masalalarda bir xil hisob-kitoblar, formulalar va algoritmlar ko'p marotaba takrorlanadi. Agar bunday amallar har safar boshqatdan yozilsa, dastur:

- juda uzun bo'lib ketadi;
- xatolarga moyil bo'ladi;
- tahrirlash va kengaytirish qiyinlashadi.

Shu sababli dasturlash tillarida qism dasturlar tushunchasi joriy etilgan. Qism dasturlar dastur kodini mantiqiy bo'laklarga ajratish, uni qayta ishlatish va optimallashtirish imkonini beradi. Python dasturlash tilida qism dasturlar asosan funksiyalar ko'rinishida amalga oshiriladi.

Mazkur mavzuda Python dasturlash tilida funksiyalar va qism dasturlarning mohiyati, ularning tuzilishi, chaqirilishi hamda muhandislik va texnik masalalarni yechishda qo'llanilishi batafsil yoritiladi. Ushbu mavzu talabalarni yirik va murakkab dasturlar yaratishga tayyorlaydi.

1. Python dasturlash tilida qism dasturlarda funksiyalarni qo'llanilishi

1.1. Qism dastur tushunchasi

Qism dastur — bu dastur ichida alohida nomga ega bo'lgan va muayyan vazifani bajaruvchi kod bo'lagi bo'lib, zarur bo'lganda chaqiriladi. Qism dasturlar:

- kodni qisqartiradi;
- takrorlanishni kamaytiradi;
- dastur tuzilishini aniq va tushunarli qiladi.

Python tilida qism dasturlar asosan **funksiyalar** orqali amalga oshiriladi. Klassik dasturlashda “protsedura” va “funksiya” tushunchalari farqlanadi. Python'da esa funksiyalar ham natija qaytaruvchi, ham natija qaytarmaydigan bo'lishi mumkin.

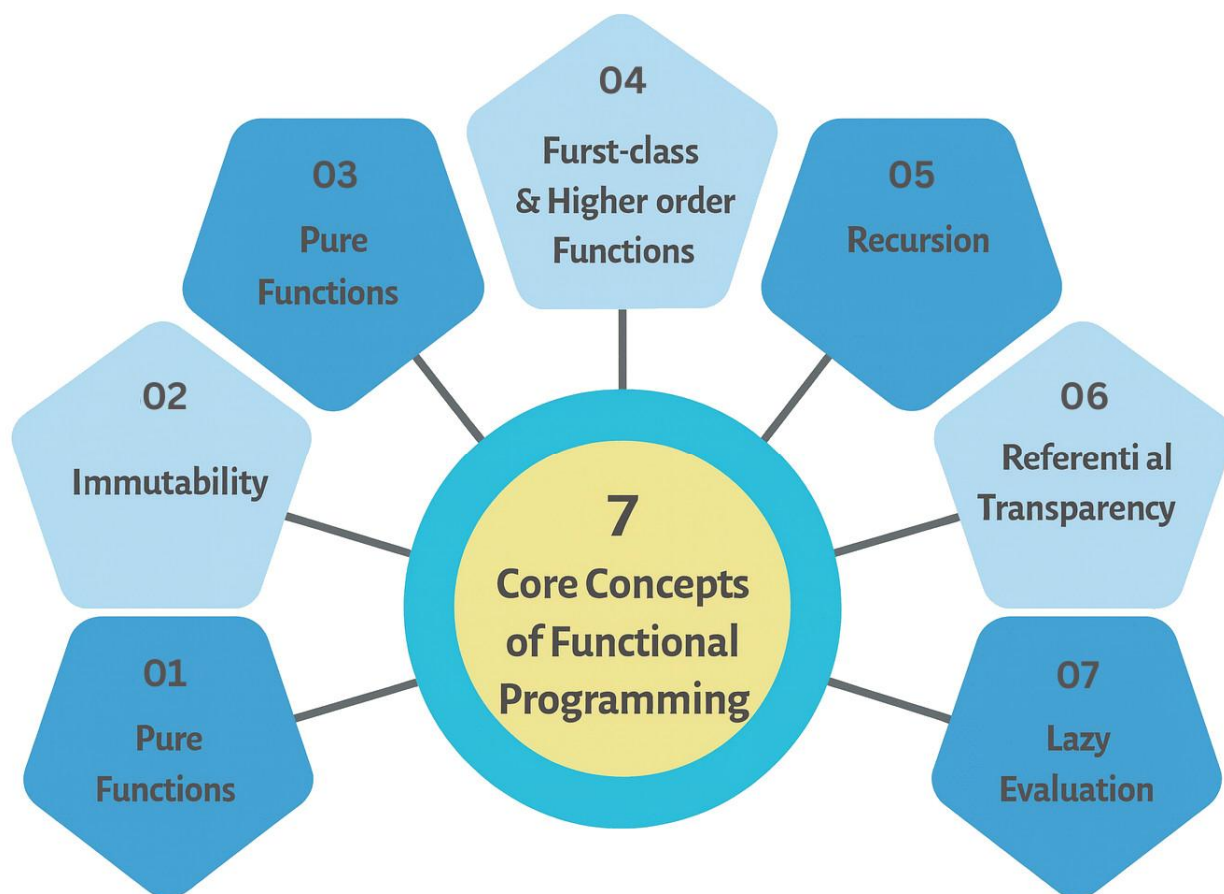
1.2. Funksiya tushunchasi

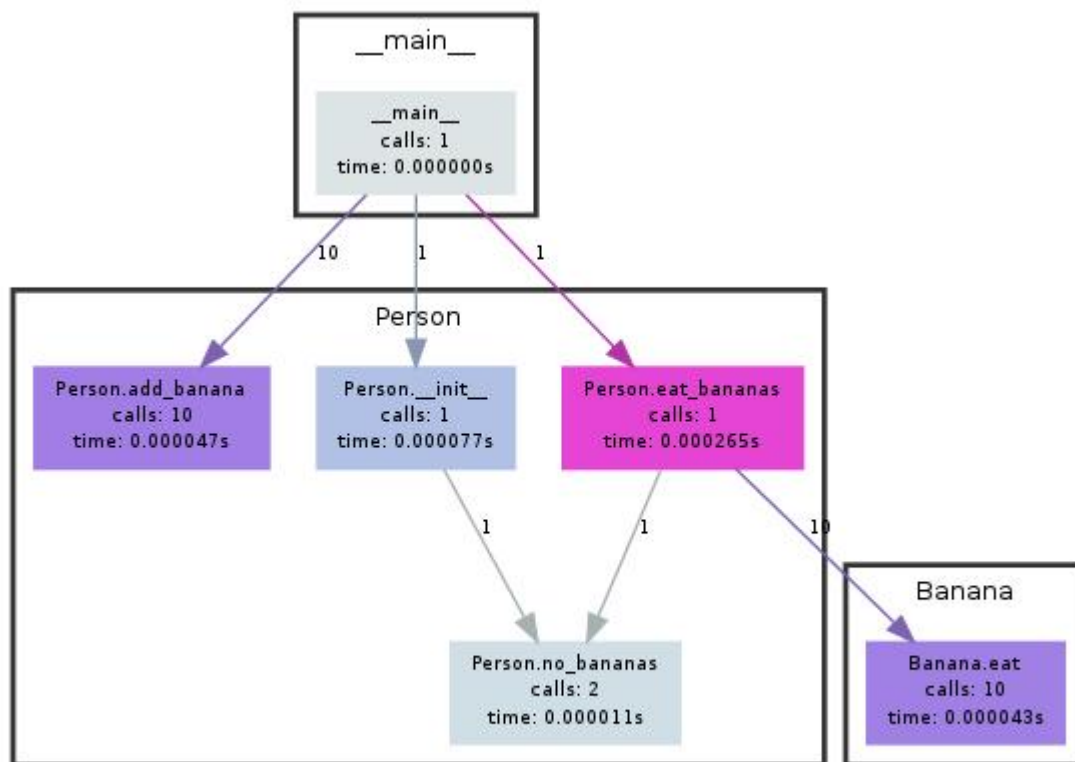
Funksiya — bu ma'lum bir amalni bajaruvchi va zarur bo'lsa natija qaytaruvchi qism dasturidir. Funksiyalar:

- parametrlar (argumentlar) qabul qilishi mumkin;
- hisoblash bajaradi;
- natijani qaytaradi yoki faqat amal bajaradi.

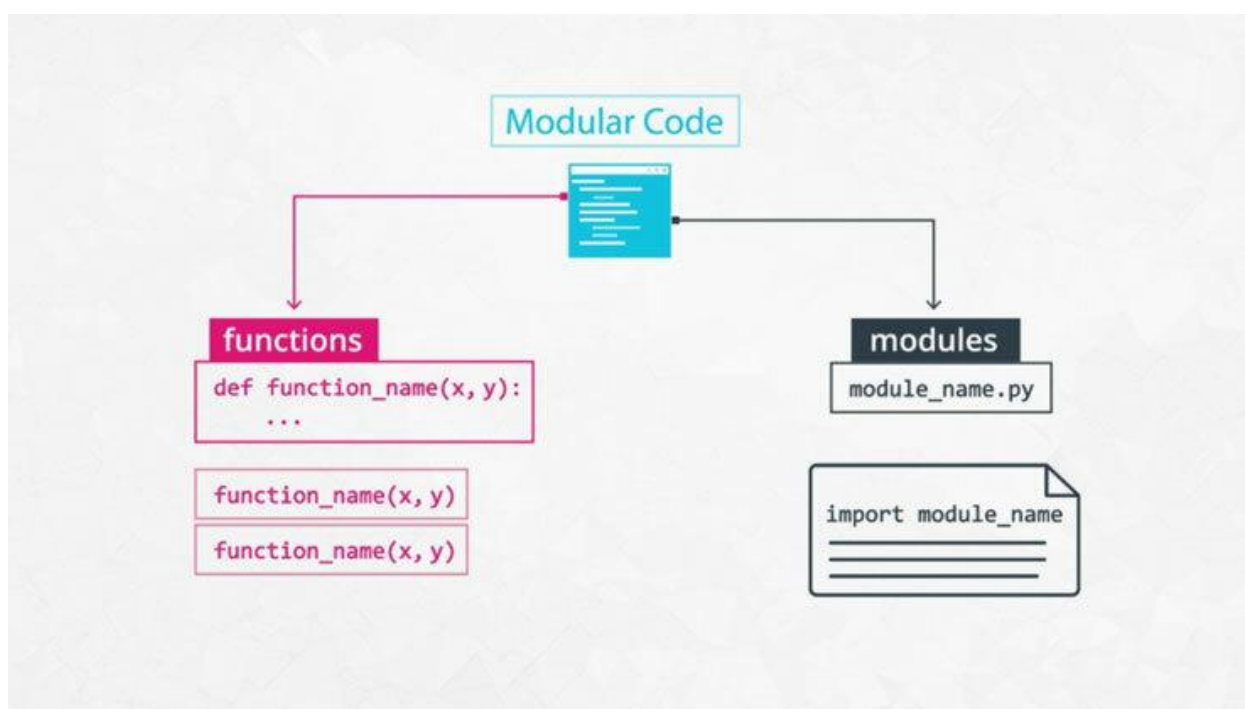
Funksiya quyidagi holatlarda juda foydali:

- bir xil hisob-kitob ko'p marta bajarilganda;
- murakkab formulalar ishlatilganda;
- algoritmnı mantiqiy qismlarga ajratish zarur bo'lganda.





Generated by Python Call Graph v1.0.0
<http://pycallgraph.slowchop.com>



Yuqoridagi rasmda dastur asosiy qismi va funksiyalar oʻrtasidagi bogʻlanish grafik koʻrinishda tasvirlangan.

1.3. Python'da funksiya tuzilishi

Python dasturlash tilida funksiya `def` kalit soʻzi yordamida aniqlanadi.

Funksiyaning umumiy tuzilishi:

```
def funksiya_nomi(parametrlar):  
    buyruqlar  
    return natija
```

Bu yerda:

- `def` — funksiya e'lon qilish operatori;
- `funksiya_nomi` — funksiya nomi;
- `parametrlar` — tashqi ma'lumotlar;
- `return` — natijani qaytarish operatori.

Oddiy misol:

```
def kvadrat(x):  
    return x * x  
  
print(kvadrat(5))
```

Bu funksiya kiritilgan sonning kvadratini hisoblaydi.

1.4. Parametrli va parametrsiz funksiyalar

Parametrsiz funksiya

```
def salomlash():  
    print("Assalomu alaykum!")  
  
salomlash()
```

Parametrli funksiya

```
def yigindi(a, b):  
    return a + b  
  
natija = yigindi(4, 6)  
print(natija)
```

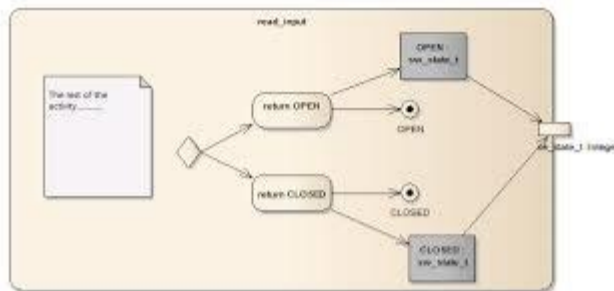
Funksiyalarni parametrli ishlatish dastur moslashuvchanligini oshiradi.

1.5. Natija qaytaruvchi va qaytarmaydigan funksiyalar

- **Natija qaytaruvchi funksiya** — `return` operatoriga ega;
- **Natija qaytarmaydigan funksiya** — faqat amal bajaradi.

```
def chiqar(x):
    print(x)
```

Python'da bu turdagi funksiyalar ko'pincha protsedura vazifasini bajaradi.



Stored Procedure (SP)	Function (UDF - User Defined Function)
SP can return zero , single or multiple values.	Function can return one value which is mandatory.
We can use transaction in SP.	We can't use transaction in UDF.
SP can have input/output parameter.	Only input parameter.
We can called function from SP.	We can't call SP from function.
We can't use SP in SELECT/ WHERE/ HAVING statement.	We can use UDF in SELECT/ WHERE/ HAVING statement.
We can use exception handling using Try-Catch block in SP.	We can't use Try-Catch block in UDF.

```
>>> def print_something(s):
    print('Printing::', s)
```

JournalDev

```
>>> output = print_something('Hi')
Printing:: Hi
>>>
>>> print(f'A function without return statement returns {output}')
A function without return statement returns None
>>>
```

2. Qism dasturlar va funksiyalarni muhandislik va texnik masalalarda qo'llash

2.1. Muhandislik masalalarida funksiyalarning ahamiyati

Muhandislik va texnik masalalarda ko'pincha:

- murakkab matematik formulalar;
- fizik qonunlar;
- takrorlanuvchi hisob-kitoblar uchraydi.

Funksiyalar yordamida bunday formulalarni alohida qism dasturga ajratish mumkin.
Bu esa:

- aniqlikni oshiradi;
- xatolikni kamaytiradi;
- dasturdan qayta foydalanishni ta'minlaydi.

2.2. Texnik hisoblashga oid misol

Masala: Jismning erkin tushish vaqti va tezligini hisoblash.

```
def tezlik(g, t):
    return g * t

def masofa(g, t):
    return 0.5 * g * t * t

g = 9.81
t = float(input("Vaqtni kiriting: "))

print("Tezlik:", tezlik(g, t))
print("Masofa:", masofa(g, t))
```

Bu misolda ikkita funksiya yordamida fizik formulalar dasturga ajratilgan.

$$\Delta x = \bar{v}t$$

$$v = v_0 + at$$

$$x = x_0 + v_0t + \frac{1}{2}at^2$$

$$x = x_0 + vt - \frac{1}{2}at^2$$

$$v^2 = v_0^2 + 2a(x - x_0)$$

2.3. Texnik tizimlarda modullashtirish

Katta texnik tizimlarda dastur quyidagilarga bo'linadi:

- kiritish moduli;
- hisoblash moduli;
- chiqarish moduli.

Har bir modul funksiya sifatida yozilishi mumkin. Bu yondashuv **modulli dasturlash** deb ataladi.

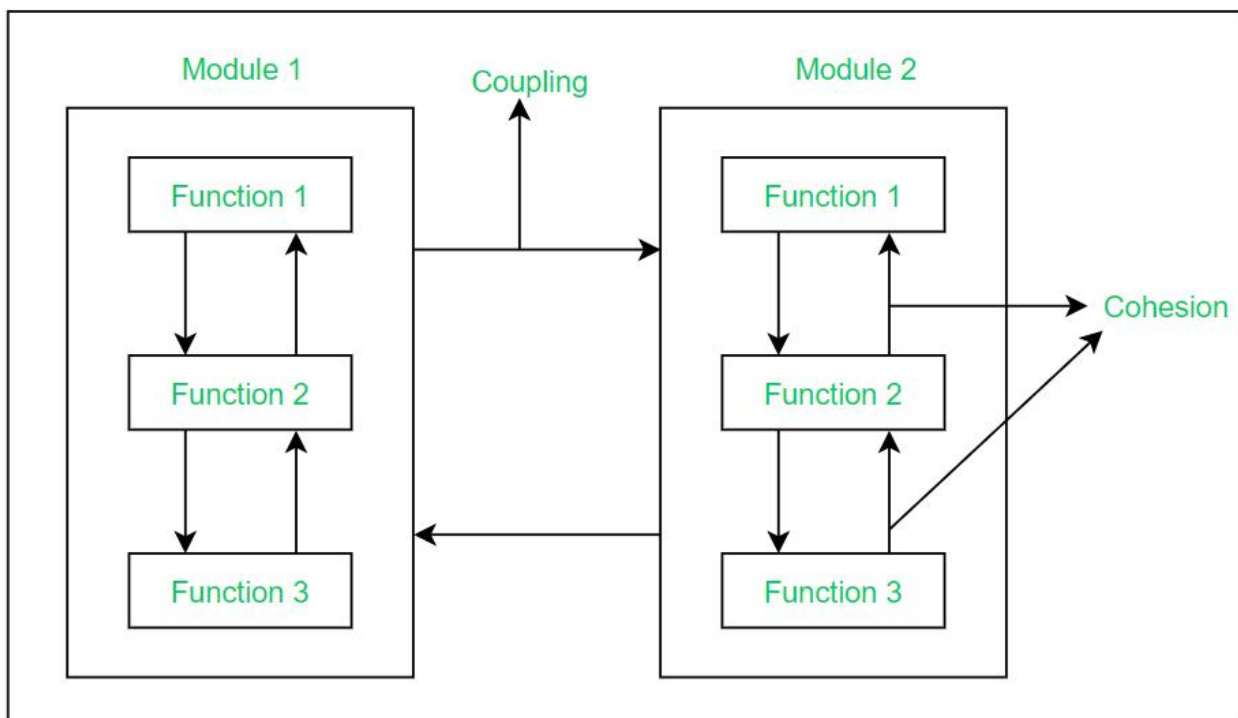
2.4. Amaliy texnik masala (elektr zanjiri)

Masala: Qarshilik, tok va kuchlanishni hisoblash.

```
def tok(u, r):  
    return u / r  
  
def quvvat(u, i):  
    return u * i  
  
u = float(input("Kuchlanishni kiriting: "))  
r = float(input("Qarshilikni kiriting: "))  
  
i = tok(u, r)  
p = quvvat(u, i)  
  
print("Tok:", i)  
print("Quvvat:", p)
```

Bu misol funksiyalar yordamida texnik hisoblashni aniq va tartibli koʻrinishda amalga oshiradi.

$$\begin{array}{ll} \nabla \cdot \vec{E} = \frac{\rho}{\epsilon_0} = 4\pi k \rho & \oint \vec{E} \cdot d\vec{A} = \frac{q}{\epsilon_0} \\ \nabla \cdot \vec{B} = 0 & \oint \vec{B} \cdot d\vec{A} = 0 \\ \nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} & \oint \vec{E} \cdot d\vec{s} = -\frac{d\Phi_B}{dt} \\ \nabla \times \vec{B} = \frac{\vec{J}}{\epsilon_0 c^2} + \frac{1}{c^2} \frac{\partial \vec{E}}{\partial t} & \oint \vec{B} \cdot d\vec{s} = \mu_0 i + \frac{1}{c^2} \frac{\partial}{\partial t} \int \vec{E} \cdot d\vec{A} \end{array}$$



```

In [5]: # Define the user prompt (prompt engineering)
...: user_prompt = "You are a tech writer for a beginner-friendly blog. Write a 300-word article explaining
...: why Python is ideal for data science in 2024. Use simple analogies (e.g., compare data to a recipe). Av
...: oid technical terms like 'pandas' or 'scikit-learn'. Keep the tone warm and encouraging. Return only th
...: e article text - no headings or markdown."

In [6]: response = client.chat.completions.create(
...:     model=MODEL_NAME,
...:     messages=[
...:         # we don't use special system message for now, only the default one
...:         {"role": "user", "content": user_prompt}
...:     ],
...:     temperature=0.35,
...:     max_tokens=300
...: )
...: print(response.choices[0].message.content)

```

Imagine you're baking a cake. You have a recipe—clear steps, ingredients, and instructions. But what if the recipe is messy, hard to follow, or changes every time you try it? That's kind of like working with raw data. It's full of numbers, patterns, and sometimes, confusion. That's where Python comes in—it's like having a well-organized kitchen with helpful tools that make cooking (or analyzing data) easier and more fun.

Python is like a friendly chef who knows how to mix, measure, and taste everything. It doesn't demand complicated tools or a fancy degree to use—it just works with what you have. Whether you're tracking sales, studying weather patterns, or predicting how many people will attend an event, Python helps you turn messy data into clear, useful answers.

One of Python's best features is that it's simple to learn. Even if you've never touched code before, you can start writing small scripts to ask questions like, "What's the average temperature this month?" or "Which product sold the most?" It's like asking your kitchen helper to organize the ingredients and show you what's working.

Plus, Python has a huge community. Think of it like a group of friends who share tips, recipes, and tricks. If you get stuck, someone's probably already solved your problem and shared it. This means you're never truly alone in your data journey.

2.5. Funksiyalarni qo'llashning afzalliklari

Funksiyalar:

- dastur hajmini kamaytiradi;
- qayta ishlatish imkonini beradi;
- testlashni osonlashtiradi;
- jamoaviy dasturlashni qulay qiladi.

Shu sababli funksiyalar zamonaviy dasturlash texnologiyalarining ajralmas qismi hisoblanadi.

Xulosa

Qism dasturlar va funksiyalar Python dasturlash tilida murakkab masalalarni samarali va tartibli yechish imkonini beradi. Muhandislik va texnik masalalarda funksiyalar yordamida formulalar, hisoblashlar va algoritmlar alohida mantiqiy bo'laklarga ajratiladi. Ushbu yondashuv dastur sifatini oshiradi va uni professional darajaga olib chiqadi. Mazkur mavzu keyingi mavzular — massivlar, fayllar va murakkab algoritmlarni o'rganish uchun mustahkam poydevor yaratadi.

Nazorat savollari

1. Qism dastur deganda nimani tushunasiz?
2. Python tilida qism dasturlar qanday amalga oshiriladi?
3. Funksiya nima va uning vazifasi nimadan iborat?
4. Funksiya qanday e'lon qilinadi?
5. Parametrli va parametrsiz funksiyalar o'rtasidagi farq nimada?
6. `return` operatorining vazifasi nimadan iborat?
7. Natija qaytaruvchi va qaytarmaydigan funksiyalar qanday farqlanadi?
8. Muhandislik masalalarida funksiyalar nima uchun muhim?
9. Modulli dasturlash nima?
10. Texnik masalalarda funksiyalardan foydalanishning afzalliklari nimada?